Logical Contracts

Sixteen rule languages, one readable language

A tour, writing rules from ordinary language, and hands-on

Miguel Calejo · Logical Contracts

JURIX 2026 tutorial · Toulouse · 8 December 2026



Open this now, for later

The lab page has the three exercises, the links, and the answers.

- A laptop with a web browser is enough
- Nothing to install
- No laptop? Work with your neighbour

The page: `LAB_URL`



the lab page

1 · A tour of sixteen rule languages, in five families

For each language: a real program, its Logical English twin, and how we checked the twin

How to read the tour

A **twin** is the translation of a program of another system into Logical English (LE), or into LE for LPS when the program changes a state over time.

- A **translator** writes each twin. The translators are programs, not a language model: the same input gives the same twin
- What the translator cannot translate stays in the twin as a **TODO** comment, with the source text
- The twin carries **scenarios** with the answers that the **source system's own engine** gave. A scenario is a set of facts, and its expected answers make it a test
- **File ▸ Show the Original...** shows the source beside the twin



Five families, sixteen languages

Family	Languages	Twin in	Checked against
Legal and academic formalisms	LegalRuleML · s(CASP) and Prolog · Blawx · Epilog · L4	LE (L4 contracts and Epilog games: LPS)	SPINdle · s(CASP) · Blawx · EpilogJS · L4's evaluator
Rules as Code for statutes, taxes, benefits	Axiom RuleSpec · PolicyEngine and OpenFisca · Catala	LE	each project's own engine or interpreter
Decision management	Drools rules (and DMN decision tables) · Oracle Intelligent Advisor	LPS, LE	a Drools session · the projects' test cases
Industry configurations, financial regulation	Socotra · Oracle Insurance Policy Administration · Rune (Digital Regulatory Reporting)	LE	reference runs · the reports that DRR expects
Ledgers and contracts	Solidity · Daml · Bitcoin Miniscript	LPS, LE	the Ethereum machine · a Canton ledger · a Bitcoin test chain



Legal and academic formalisms 1 of 5

LEGALRULEML (OASIS STANDARD), SECTION 29 OF AN AUSTRALIAN CREDIT ACT

```
<lrml:PrescriptiveStatement key="ps1">
 <ruleml:Rule key=":rule1" closure="universal">
  <ruleml:if> … <ruleml:Rel iri=":person"/> …
  <ruleml:then> … <lrml:Prohibition> …
     <ruleml:Rel iri=":engageCreditActivity"/> …
```

THE TWIN, IN LOGICAL ENGLISH

```
rule ps1:
a party is forbidden that the party engages in a credit activity if
    the party is a person
    and it is not the case that the party holds a credit licence.
```

Language	Checked
LegalRuleML	SPINdle, 68 of 68
s(CASP), Prolog	s(CASP), 23 of 23; round trip gives the same program for 87 of 93
Blawx	Blawx, 163 of 163 in 15 projects
Epilog	EpilogJS, 21 of 21; games 71 of 75
L4	L4's evaluator, 621 of 621 rules



Rules as Code for statutes, taxes and benefits 2 of 5

AXIOM RULESPEC, THE US FOOD STAMPS (SNAP) MAXIMUM ALLOTMENT

```
formula: |-
  snap_maximum_allotment_table[max(min(household_size, 8), 1)]
  + (max(household_size - 8, 0) * snap_maximum_allotment_additional_member)
```

THE TWIN

```
the snap maximum allotment of a household is an amount N if
    … and the minimum of M and 8 is a number K
    and the maximum of K and 1 is a number P
    and the snap maximum allotment table for P is an amount Q …
    and N = Q + T * R.
```

Language	Checked
Axiom RuleSpec	Axiom's engine, 473 of 473; written back, 37 of 39 pass, 2 refused
PolicyEngine, OpenFisca	the packages' own engines; French income tax 2,286 of 2,302
Catala	Catala's interpreter, 129 of 129; written back, 42 of 42

Catala twins also exist in **Français Logique**, the French variant of LE.



Decision management and business rules 3 of 5

DROOLS, A RULE OF ITS INSURANCE EXAMPLE

```
rule "young drivers are high risk"
  when Driver( $name : name, age == "young" )
       not Policy( driver == $name )
  then insert(new Policy($name, "high"));
```

THE TWIN, IN LOGICAL ENGLISH FOR LPS

```
if there is a driver called a name with age young and record a record
    and it is not the case that
        there is a policy for the name with band a band
then a policy starts for the name with band high.
```

- Drools changes a **working memory**. The twin changes **the world**: `insert` is an event, with a causal law
- When the rules only insert facts, the translator also writes a **decision** with no time
- **DMN** decision tables stay tables in LE, with their hit policy

Language	Checked
Drools	a Drools session, same final memory 5 of 5
DMN	the test suite of DMN, 138 of 138; the way back, 424 of 424
Oracle Intelligent Advisor	the projects' tests, 953 of 953 in 15 projects

Industry configurations and financial regulation 4 of 5

ORACLE INSURANCE POLICY ADMINISTRATION, A "MATH VARIABLE"

```
<MathVariable VARIABLENAME="InGraceMV" TYPE="IIF"
  EXPRESSION="DaysPastDueMV > 0 And DaysPastDueMV <= GracePeriodDaysMV">
  YesMV,NoMV
```

THE TWIN

```
rule in_grace with provenance "GraceCheck/Transaction.xml"
                          at MathVariable InGraceMV:
the in grace of an activity is a value if
    the days past due of the activity is a number N
    and the grace period days of the plan is a number M
    and N > 0 and N <= M and the value is equal to "Yes"
    otherwise … the value is equal to "No".
```

Language	Checked
Socotra	a reference run of its scripts, 992 of 992
OIPA	an independent interpreter of its maths, 103 of 103
Rune (DRR)	the reports DRR expects: 99.1% of 102,879 fields; written back, 2,197 of 2,249 reports identical

Rune is the language of the derivatives industry's **Digital Regulatory Reporting**: what a bank must report to a regulator, for each trade.



Ledgers and contracts 5 of 5

BITCOIN MINISCRIPT, AN INHERITANCE WALLET (LIANA)

```
or(99@pk(manager),
   or(9@and(thresh(2,pk(spouse),pk(first_child),pk(second_child)),
            older(52560)),
      and(pk(third_party),older(65535))))
```

THE TWIN

```
the coin can be spent if
    a number N is the count of each K such that
        a key K is in [the spouse, the elder child, the younger child]
        and the witness contains a signature by K
    and N >= 2
    and the coin has been confirmed for a number M blocks
    and M >= 52560.
```

Language	Twin	Checked
Solidity	LPS	the Ethereum machine, a USD Coin history 849 of 849
Daml	LPS	a Canton ledger; written back, 26 the same, 17 refused, 0 different
Miniscript	LE	a Bitcoin test chain, 361 of 361

- **Refusal is a feature**: when the way back cannot keep the meaning, the exporter says no



Demo · A twin, live

1. **LE2**: open `migration/legalruleml/ex5_section29new/ex5_section29new`
2. **File ▸ Show the Original...** — the LegalRuleML beside the twin
3. Scenario `unlicensed_trader`, query `obligations`: Alice's two penalties, and **why**
4. Scenario `licensed_trader`, query `prohibitions`: no answer, and **why not**
5. **LPS2**: open `migration/drools/fire_alarm/fire_alarm.le`, **Run**, the **timeline**

Script: demos/tour_demo.md · backup: the screenshots in demos/



What the tour shows

- The **same few ideas** come back in every family: conditions, exceptions, amounts, tables, obligations, events
- One readable language can carry all of them, **with the source beside it**
- A twin is **checked** against the source system's engine. A difference is a finding, about the twin or about the source
- Some twins go **both ways**: RuleSpec, Catala, DMN, Rune, Daml, Miniscript, and Solidity from LPS



2 · Writing rules from ordinary language

A language model proposes, the engine checks, the scenarios decide

Three steps, and who is responsible for each

1 · THE MODEL PROPOSES

A large language model reads the text and writes Logical English: templates, rules, a scenario.

2 · THE ENGINE CHECKS

The LE **verifier** reads the draft and reports each problem, for example a sentence that matches no template. The model must correct each problem.

3 · THE SCENARIOS DECIDE

You write the cases, with the answers that the text gives. The program must give those answers.

- The model **never gives the answer**. The program gives it, each time the same
- A good scenario comes from **the text and from a person**, not from the model that wrote the rules
- Each answer has an **explanation**: the rules and facts that make it true

> A language model is good at language. A logic program is a good place to keep rules.

Where the model works, in the editor

Tool	Where	What it writes
LE Assistant	the bottom panel of the LE2 editor	it reads, writes, checks and runs the program that is open
Write it in English	the Scenario Editor and the Query Editor	facts or a question, with the templates of the program
Contract Assistant	a separate page	a whole program from a contract or a regulation: several drafts, the scenarios choose

Set the model and its key: **Misc ▸ API Keys & Assistant Settings…** The model can be from OpenAI, Anthropic, Google and others.



Demo · From a regulation to a tested program

EU Regulation 261/2004, Article 7(1): compensation for a cancelled flight

1. Paste the article into the **LE Assistant**: *write the rules, and one scenario for each band*
2. Read the draft; the verifier's messages; the model's corrections
3. **Misc ▸ Run the Program's Tests…** — then change 1 500 km to 1 501 km in a scenario
4. The same article **in French**, into **Français Logique**

Script and texts: demos/writing_demo.md · compare with the example `regulatory/eu261_integration`



The keywords are translated, not only the words

```
a person acquires British citizenship on a date
if the person is born in the UK on the date
    and the date is after commencement …
```

```
une personne acquiert la citoyenneté britannique à une date
si la personne naît au Royaume-Uni à la date
    et la date est postérieure à l'entrée en vigueur …
```

```
uma pessoa adquire cidadania britânica em uma data
se a pessoa nasceu em o Reino Unido em a data …
```

```
una persona adquiere ciudadanía británica en una fecha
si la persona nació en el Reino Unido en la fecha …
```

```
una persona acquisisce la cittadinanza britannica in una data
se la persona nasce in il Regno Unito in la data …
```

- English, French, Portuguese, Spanish, Italian: one engine, five grammars
- A program says its language in its first sentence: `la langue cible est : prolog.`
- Answers and explanations come in the language of the program



3 · Hands-on

Twenty minutes. We walk around the room: raise a hand



The three exercises

1. **A LegalRuleML twin** · 6 minutes
   Open it, read the original, ask *why* and *why not*
2. **A Drools twin** · 6 minutes
   Run the fire alarm, read the timeline, ask which rule made the alarm go on
3. **Break a rule** · 8 minutes
   Remove one condition from section 29, run the tests, and explain which scenarios fail

More, if you finish: open your own LegalRuleML or Drools file · write a rule in French · ask the LE Assistant

the lab page: steps, links, answers

le2.logicalcontracts.com

What to take home

- **Sixteen languages, five families**, one readable language between them
- Each twin is **checked** against the source system, and keeps the source beside it
- A language model **drafts**; the engine and **your scenarios** decide
- All of it runs in a browser, **open source** under the Apache 2.0 licence



the source code

le2.logicalcontracts.com · lps2.logicalcontracts.com · github.com/LogicalContractsOrg